

Защита REST API и валидация пользовательских данных

express-validator, request limits, Helmet, rate limiting и безопасный middleware pipeline

Результаты занятия

- ✓ Объяснять, почему любой пользовательский ввод считается недоверенным
- ✓ Различать client-side и server-side validation
- ✓ Различать syntactic и semantic validation
- ✓ Использовать express-validator для body, params и query
- ✓ Формировать единый ответ при validation error
- ✓ Извлекать только проверенные данные через `matchedData()`
- ✓ Ограничивать размер request body
- ✓ Использовать Helmet для security headers
- ✓ Использовать rate limiting для API и login endpoint
- ✓ Строить многоуровневый security middleware pipeline

Учебные вопросы

- 1 Недоверенные данные** Server-side validation, allowlist и бизнес-ограничения
- 2 express-validator** body, param, query, validationResult и matchedData
- 3 Безопасный input pipeline** Validation, normalization, unknown fields и error response
- 4 HTTP hardening** Request-size limits и Helmet security headers
- 5 Rate limiting и defence in depth** 429, login limiter, logging и итоговый middleware pipeline



Учебный вопрос 1. Недоверенные данные

Любой HTTP input может быть изменён вне интерфейса frontend

Главное правило: не доверять клиенту

- HTML input, select и disabled button не являются механизмами безопасности.
- Пользователь может отправить запрос через DevTools, Postman, curl или собственный script.
- Можно изменить body, params, query и headers.
- Поэтому backend валидирует все данные до бизнес-логики и работы с базой.

Client-side и Server-side validation

Client-side

- Улучшает UX
- Даёт быстрый feedback
- Может использовать required/min/max
- Легко обходится
- Не является security boundary

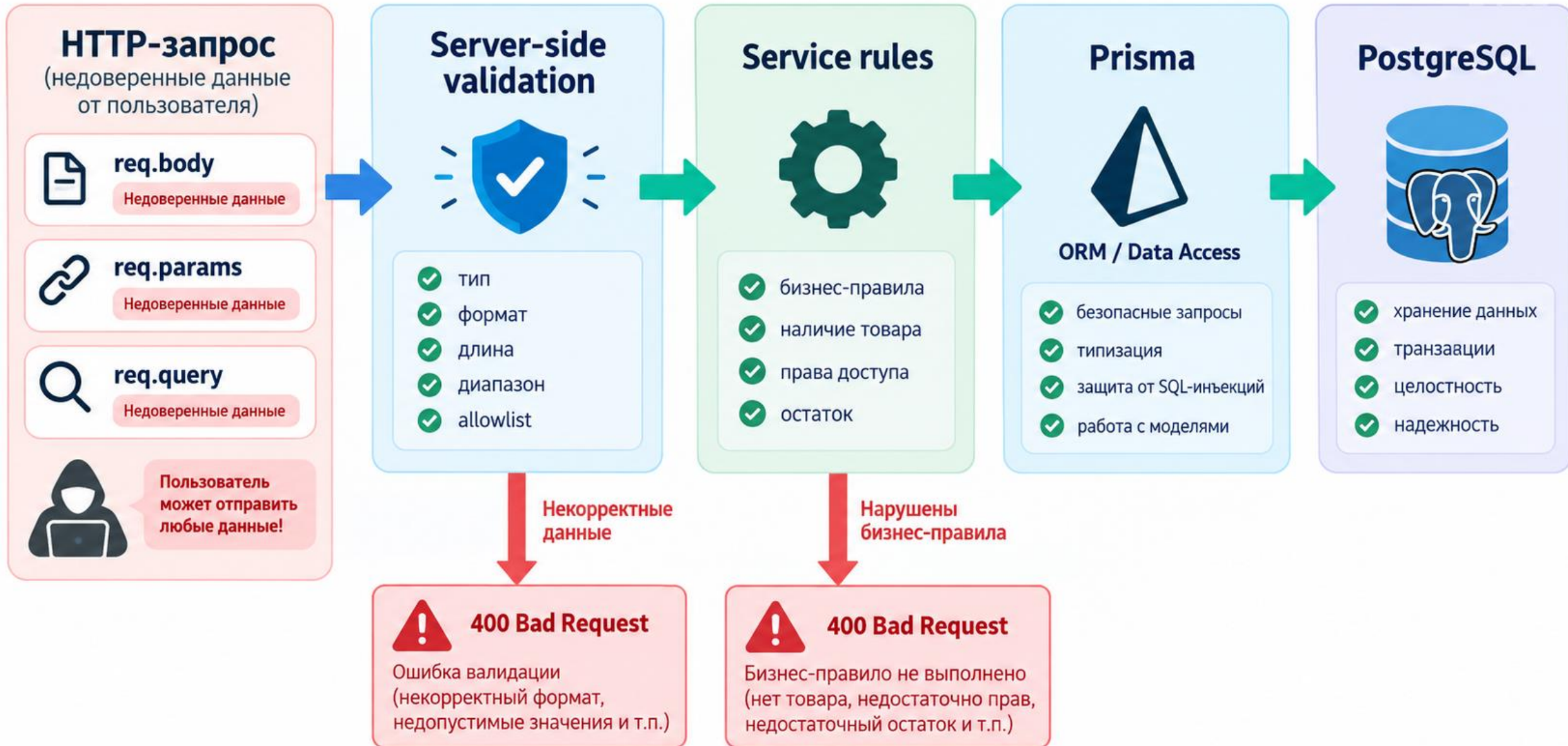
Server-side

- Обязательна для безопасности
- Проверяет фактически полученный HTTP request
- Работает независимо от frontend
- Останавливает ошибочные данные до Service/DB
- Является частью API contract

Syntactic и semantic validation

- Syntactic validation проверяет форму данных: тип, формат, длину.
- Semantic validation проверяет смысл в бизнес-контексте.
- Пример syntax: price должен быть integer.
- Пример semantics: quantity не может превышать допустимое число единиц товара.
- Обе проверки нужны и решают разные задачи.

Недоверенные данные → валидация → бизнес-правила → база данных



Allowlist лучше denylist

- Allowlist определяет, какие значения разрешены.
- Например `role ∈ user/admin` или `sort ∈ price/name`.
- Denylist пытается перечислить опасные значения и легко обходится.
- Для фиксированных наборов и структурированных полей используйте allowlist.
- Validation не заменяет parameterized queries и безопасные API.

Что проверять в Internet Shop

Поле	Пример правила	Тип проверки
name	2–150 символов	Длина и тип
price	Целое число > 0	Тип и диапазон
categoryId	Integer >= 1	Тип и диапазон
inStock	Boolean	Тип
quantity	1–100	Диапазон
role	user или admin	Allowlist
id в URL	Integer >= 1	Path parameter



Учебный вопрос 2. express-validator

Validation rules удобно оформлять middleware-цепочками

Установка express-validator

BASH

```
npm install express-validator
```

Основные функции

- `body()` проверяет поля `req.body`.
- `param()` проверяет параметры маршрута.
- `query()` проверяет `query string`.
- `validationResult(req)` получает накопленные `validation errors`.
- `matchedData(req)` извлекает только данные, прошедшие `validation/sanitization`.

Validation Product при POST

JAVASCRIPT

```
import { body } from "express-validator";

export const createProductValidation = [
  body("name")
    .isString()
    .trim()
    .isLength({ min: 2, max: 150 }),

  body("price")
    .isInt({ min: 1, max: 100000000 })
    .toInt(),

  body("categoryId")
    .isInt({ min: 1 })
    .toInt(),

  body("inStock")
    .optional()
    .isBoolean()
    .toBoolean()
];
```

Что важно

- Каждое поле получает независимую validation chain.
- trim() нормализует name.
- toInt() и toBoolean() преобразуют валидные значения к ожидаемому типу.
- optional() разрешает отсутствие необязательного поля.

Validation chain выполняется слева направо

- Validator проверяет значение.
- Sanitizer может нормализовать или преобразовать значение.
- `withMessage()` позволяет задать понятное сообщение.
- `bail()` может остановить дальнейшие проверки поля после первой ошибки.
- Контроллер запускается после `validation middleware`.



Validation flow in Express: body / param / query → validationResult

Данные из разных источников (req.body, req.params, req.query) проходят через цепочки проверок **express-validator**.
Результат проверяется с помощью validationResult(req).

1. Источники данных запроса



req.body
Тело запроса (JSON)

```
{
  "name": "Ноутбук",
  "price": 999,
  "categoryId": 1
}
```



req.params
Параметры маршрута (URL)

/api/products/123

req.params.id = "123"



req.query
Строка запроса (Query String)

/api/products?page=1&limit=10

```
req.query = {
  "page": "1",
  "limit": "10"
}
```

2. Цепочки валидации (express-validator)



express-validator
Проверяем, валидируем и санитизируем данные

Валидация тела запроса (body)

```
body('name')
  .isString().notEmpty().withMessage('Название обязательно')

body('price')
  .isFloat({ min: 0 }).withMessage('Цена должна быть > 0')

body('categoryId')
  .isInt({ min: 1 }).withMessage('Некорректная категория')
```

Валидация параметров (param)

```
param('id')
  .isInt({ min: 1 }).withMessage('Некорректный ID')
```

Валидация строки запроса (query)

```
query('page')
  .optional().isInt({ min: 1 }).withMessage('page > 0')

query('limit')
  .optional().isInt({ min: 1, max: 100 }).withMessage('limit 1-100')
```

3. Проверка результата

```
import { validationResult }
from 'express-validator';

app.post('/api/products',
[
  // цепочки валидации
  body('name').isString().notEmpty(),
  body('price').isFloat({ min: 0 }),
  param('id').isInt({ min: 1 }),
  query('page').optional().isInt(),
], async (req, res) => {

  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({
      message: 'Validation failed',
      errors: errors.array()
    });
  }

  // данные валидны – продолжаем
  next();
});
```

4a. Успешная валидация



errors.isEmpty() === true
Данные прошли все проверки.



Дальше обработчик

- ✓ req.body (валидные данные)
 - ✓ req.params (валидные данные)
 - ✓ req.query (валидные данные)
- Продолжаем выполнение (**next()**)

4b. Ошибки валидации



errors.isEmpty() === false
Возвращаем 400 Bad Request с описанием ошибок.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "message": "Validation failed",
  "errors": [
    {
      "type": "field",
      "msg": "Название обязательно",
      "path": "name",
      "location": "body"
    }
  ]
}
```



Преимущества такого подхода

- ✓ Единый способ валидации для body, params и query.
- ✓ Подробные и понятные сообщения об ошибках.
- ✓ Можно выполнять санитизацию данных.
- ✓ Легкая интеграция в middleware цепочку.



Важно помнить

- Валидация на сервере обязательна (client-side — только для удобства).
- Проверяйте только необходимые поля (allowed list).
- Используйте понятные сообщения об ошибках.
- Не передавайте дальше необработанные данные.



Результат

Надёжная проверка входящих данных, меньше ошибок и более безопасное API.

Единый validateRequest middleware

JAVASCRIPT

```
import {
  matchedData,
  validationResult
} from "express-validator";

export function validateRequest(req, res, next) {
  const result = validationResult(req);

  if (!result.isEmpty()) {
    return res.status(400).json({
      error: "Validation failed",
      details: result.array()
    });
  }

  req.validated = matchedData(req, {
    includeOptionals: true
  });

  next();
}
```

Что важно

- Все validation errors собираются в одном месте.
- Некорректный request завершается status 400.
- matchedData() формирует набор только проверенных полей.
- Service получает данные после validation.

Подключение validator к route

JAVASCRIPT

```
router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  createProductValidation,
  validateRequest,
  createProduct
);
```

Что важно

- Authentication выполняется до admin action.
- Role проверяется до Controller.
- Validation выполняется до бизнес-логики.
- Controller получает уже проверенный request.

Validation path parameter

- URL parameter тоже является пользовательским вводом.
- Не достаточно вызвать `Number(req.params.id)` и надеяться на корректность.
- Проверяйте диапазон и тип перед запросом к базе.
- Например `id` должен быть положительным integer.

Проверка :id

JAVASCRIPT

```
import { param } from "express-validator";

export const idValidation = [
  param("id")
    .isInt({ min: 1 })
    .withMessage("id must be a positive integer")
    .toInt()
];
```



Учебный вопрос 3. Безопасный input pipeline

Проверяем не только формат, но и то, какие поля вообще разрешены

Mass assignment / unexpected fields

- Клиент может прислать больше полей, чем показывает UI.
- Например register body может содержать role: admin.
- Нельзя слепо передавать req.body целиком в prisma.create() или update().
- Используйте явный список разрешённых полей или matchedData().

Опасно и безопасно

Опасно

- data: req.body
- Клиент управляет неожиданными полями
- Сложнее увидеть реальный API contract
- Риск изменения role/status/owner

Безопаснее

- data: req.validated
- Передаются только проверенные поля
- Contract явно определён validators
- Системные поля задаёт backend

Controller использует req.validated

JAVASCRIPT

```
export async function createProduct(req, res, next) {  
  try {  
    const product = await addProduct(req.validated);  
    res.status(201).json(product);  
  } catch (error) {  
    next(error);  
  }  
}
```

Что важно

- Controller не читает сырой req.body.
- req.validated сформирован middleware.
- Service получает только ожидаемые данные.

Raw request → validators → matchedData → Controller → Service

Проверяем входные данные. В сервис и базу попадают только разрешённые поля.

Raw request

Входящий HTTP-запрос от клиента (req.body)

```
POST /api/products
Content-Type: application/json

{
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "inStock": true,
  "role": "admin"
}
```

Клиент может прислать любые поля, в том числе неожиданные.

Validators

express-validator

Проверяем разрешённые поля и запрещаем неожиданные.

```
const { body, checkExact } =
  require('express-validator')
const productValidators = [
  body('name').isString()
    .notEmpty(),
  body('price').isInt({ min: 0 }),
  body('categoryId').isInt(),
  body('inStock').isBoolean(),
  checkExact() // только эти поля
]
```

- ✓ Проверяем: name, price, categoryId, inStock
- ✓ Не принимаем неизвестные поля (checkExact / allowlist)

matchedData(req)

Получаем только проверенные поля из запроса.

```
req.validated = {
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "inStock": true
}
```

✓ Только проверенные поля.

Неожиданные поля (например, role) не попадают в req.validated.

Controller

Controller использует req.validated, а не req.body.

```
async function
createProduct(req, res) {
  const data = req.validated

  const product =
    await productService.
      create(data)

  res.status(201).json({
    id: product.id
  })
}
```

Service

В Service и Prisma не попадают неожиданные поля.

```
async function create(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock
    }
  })
}
```



В базу данных попадают только разрешённые поля.



"role": "admin"

Неожиданное поле

Отклоняется и не попадает дальше в обработку.



Клиент не должен протаскивать role: "admin" в Prisma

Не доверяйте данным от клиента.
Всегда валидируйте и используйте только проверенные поля (req.validated / matchedData).

Validation Order items

JAVASCRIPT

```
export const createOrderValidation = [  
  body("items")  
    .isArray({ min: 1, max: 50 }),  
  
  body("items.*.productId")  
    .isInt({ min: 1 })  
    .toInt(),  
  
  body("items.*.quantity")  
    .isInt({ min: 1, max: 100 })  
    .toInt()  
];
```

Что важно

- items должен быть массивом.
- Количество элементов запроса ограничивается.
- Wildcard items.* проверяет каждый элемент.
- quantity получает явный диапазон.

Validation не заменяет Service rules

- Validator может проверить quantity от 1 до 100.
- Но наличие Product проверяется через Prisma.
- Достаточный остаток товара — бизнес-правило.
- Право пользователя на действие — Authorization.
- Database constraints остаются последним защитным слоем.

Слои проверки данных

Слой	Что проверяет	Пример
Validator	Форма входных данных	price — positive integer
Auth middleware	Identity и role	admin required
Service	Бизнес-правила	Product существует
Prisma/DB	Целостность	FK, UNIQUE, NOT NULL, CHECK



Учебный вопрос 4. HTTP hardening

Даже валидный JSON не должен быть неограниченного размера

Ограничение request body

- Большой body расходует память и CPU ещё до бизнес-логики.
- Express JSON parser поддерживает limit.
- При превышении body-size limit запрос отклоняется.
- OWASP рекомендует задавать разумный максимальный размер request.
- Для обычного Internet Shop API десятки килобайт обычно более чем достаточно.

ЯВНЫЙ JSON body limit

JAVASCRIPT

```
app.use(express.json({  
  limit: "100kb"  
}));
```

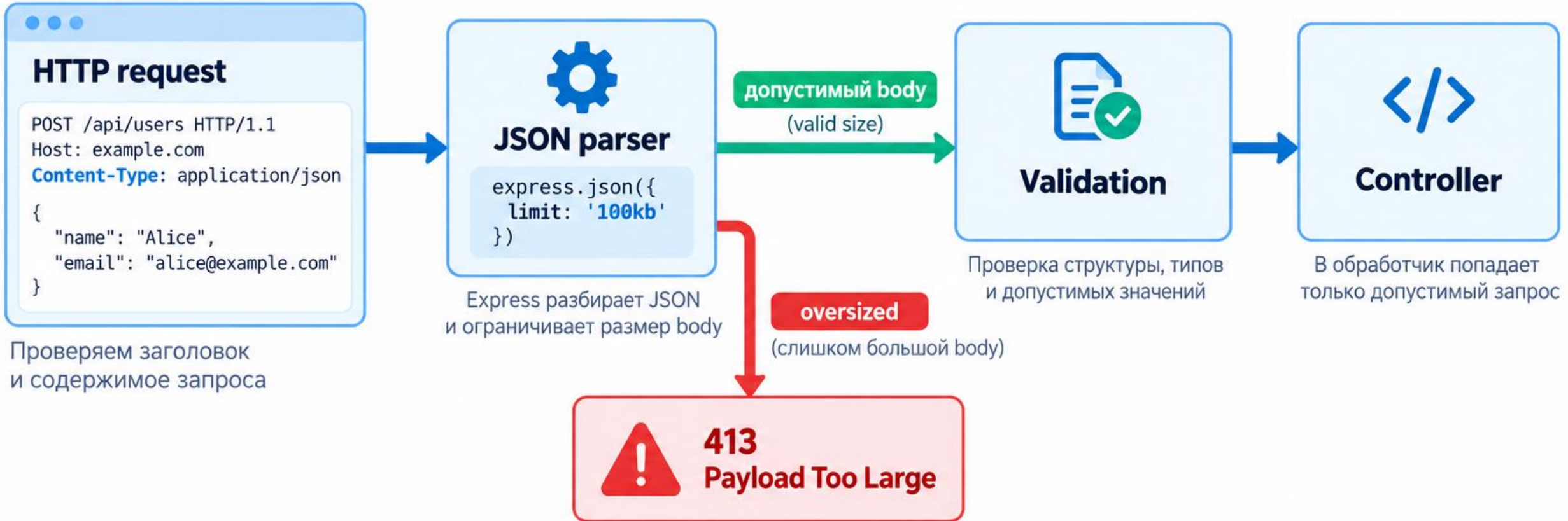
413 Payload Too Large

- Если body превышает configured limit, parser отклоняет request.
- HTTP 413 означает слишком большой request content.
- Это уменьшает риск случайной или намеренной перегрузки приложения.
- Limit должен быть числовым или корректной строкой размера и не должен зависеть от недоверенного ввода.

Request hardening: Content-Type + size limit + validation

Express.js REST API — безопасная обработка входящих запросов

Express.js 



Идея:

Слишком большой request отклоняется раньше.
Допустимый JSON проходит дальше: **parser** → **validation** → **controller**.



Безопасные
API — надёжные
приложения

Установка Helmet

BASH

```
npm install helmet
```

Helmet в Express

JAVASCRIPT

```
import helmet from "helmet";  
  
app.use(helmet());
```

Что важно

- Helmet устанавливает набор защитных HTTP response headers.
- В том числе Content-Security-Policy и другие browser security headers.
- Helmet также удаляет X-Powered-By.
- Конкретная CSP может потребовать настройки под frontend приложения.

Что дают security headers

- Content-Security-Policy уменьшает риск ряда XSS-сценариев.
- X-Content-Type-Options: nosniff отключает MIME sniffing.
- Referrer-Policy ограничивает передачу Referer.
- Strict-Transport-Security заставляет browser предпочитать HTTPS после получения policy.
- Security headers являются defense in depth, а не заменой validation.



Учебный вопрос 5. Rate limiting и defence in depth

Ограничиваем частоту запросов и защищаем чувствительные endpoints

Установка express-rate-limit

BASH

```
npm install express-rate-limit
```

Общий API limiter

JAVASCRIPT

```
import { rateLimit } from "express-rate-limit";

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 300,
  standardHeaders: true,
  legacyHeaders: false
});

app.use("/api", apiLimiter);
```

Что важно

- windowMs задаёт окно подсчёта.
- limit задаёт число разрешённых запросов на client key за окно.
- standardHeaders включает стандартные RateLimit headers.
- После превышения лимита сервер возвращает 429.

Rate limiting в Express.js: защита от слишком частых запросов

Обычные пользователи получают доступ к API. При превышении лимита запросов возвращается 429 Too Many Requests. Для чувствительных маршрутов (например, /auth/login) используется более строгий лимитер.



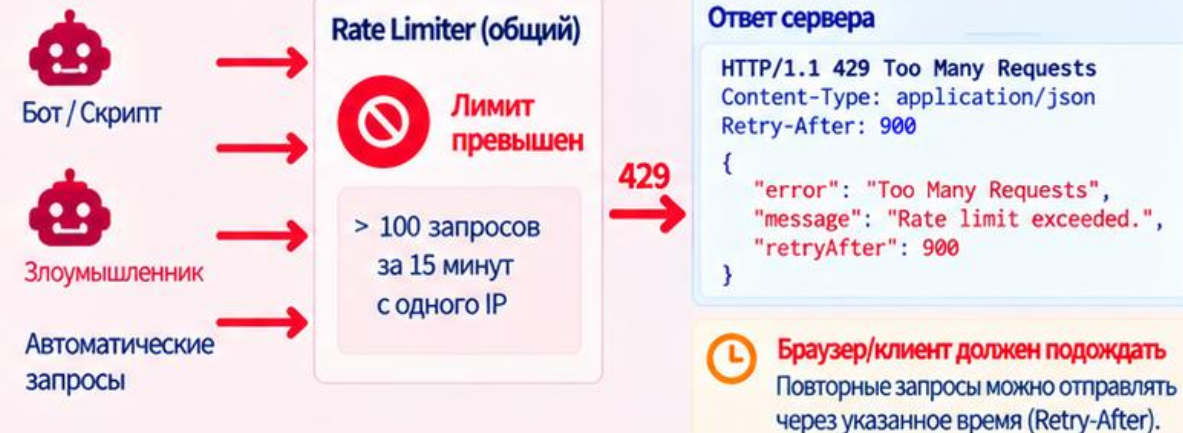
Internet Shop

Learn. Build. Deploy.

1. Обычный трафик — доступ разрешён



2. Слишком много запросов — блокировка



3. Более строгий лимитер для /auth/login



Дополнительные меры для /auth/login

- ✓ Строгий rate limit (меньше запросов).
- ✓ Постепенная блокировка (экспоненциальная задержка).
- ✓ CAPTCHA после нескольких неудачных попыток.
- ✓ Логирование подозрительной активности.
- ✓ Возможность временной блокировки IP.



Пример настройки (express-rate-limit)

```
import rateLimit from 'express-rate-limit';
// Общий лимитер
export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 минут
  max: 100, // 100 запросов
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too Many Requests' }
});
```

Строгий лимитер для /auth/login

```
export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 минут
  max: 5, // 5 попыток
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too Many Login Attempts' }
});
```

Применение в маршрутах

```
import { apiLimiter, loginLimiter } from './middleware/limit.js';

app.use('/api', apiLimiter);
app.post('/auth/login', loginLimiter, loginController);
```



Результат

- ✓ API защищён от злоупотреблений.
- ✓ Стабильная работа сервера.
- ✓ Снижение нагрузки.
- ✓ Более безопасная авторизация.
- ✓ Лучший пользовательский опыт для легитимных пользователей.

Отдельный limiter для login

JAVASCRIPT

```
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 10,
  standardHeaders: true,
  legacyHeaders: false
});

router.post(
  "/login",
  loginLimiter,
  loginValidation,
  validateRequest,
  login
);
```

Что важно

- Login является чувствительным endpoint.
- Более строгий лимит усложняет массовый online guessing.
- Rate limit дополняет password hashing, но не заменяет его.

Rate limiting: важные ограничения

- Один универсальный лимит не подходит всем endpoints.
- За reverse проху нужно правильно настроить client IP/trust проху.
- In-memory store подходит для одной учебной instance, но не для распределённого production.
- Rate limiting — защита от abuse, а не полноценная DDoS-защита.

Security pipeline Internet Shop



CORS policy



Helmet



Request body size limit



Rate limiting



Authentication



Authorization



Input validation



matchedData / explicit fields



Service business rules



Prisma + database constraints



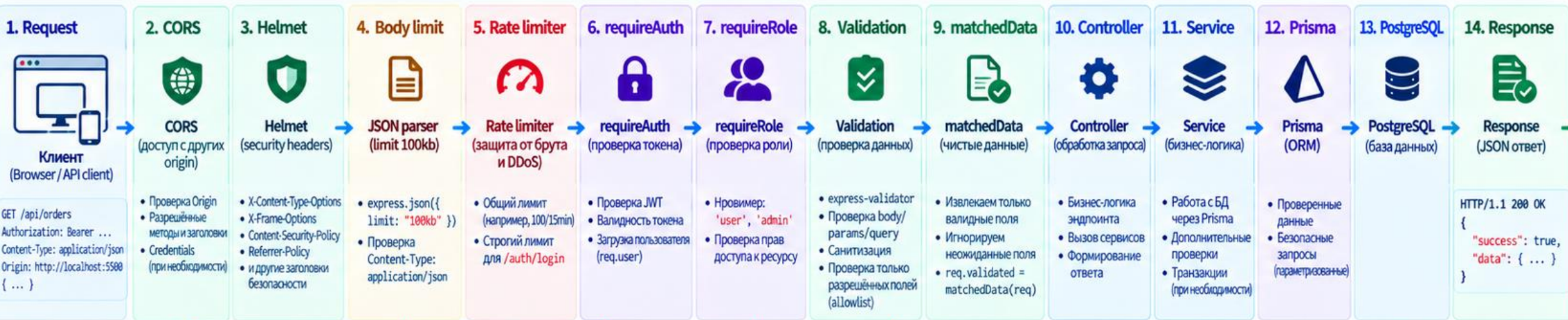
Safe error responses and security logging

Security middleware pipeline в Express.js (REST API)

Последовательная обработка запроса: от входящего HTTP-запроса до ответа. На каждом этапе возможна остановка с соответствующим кодом ошибки.



Internet Shop
Learn. Build. Deploy.



Порядок middleware в Express.js

- cors() — разрешение кросс-доменных запросов
- helmet() — security headers
- express.json({ limit: "100kb" }) — ограничение размера тела
- rateLimit() — ограничение количества запросов
- requireAuth() — проверка токена
- requireRole() — проверка роли
- validation chains — проверка входящих данных
- matchedData() — оставляем только валидные поля
- controller → service → prisma → database

Преимущества такого подхода

- Многоуровневая защита (defense in depth)
- Ранняя остановка вредоносных запросов
- Чистые и проверенные данные в контроллере
- Защита от брута, DDoS и больших payload
- Контроль доступа на основе ролей
- Минимизация риска инъекций и утечки данных
- Более стабильное и безопасное API

Рекомендации

- Размещайте middleware в правильном порядке
- Используйте allowlist валидацию (matchedData)
- Устанавливайте разумные лимиты (size, rate)
- Применяйте строгие правила для /auth/login
- Логируйте ошибки и подозрительную активность
- Используйте HTTPS в production
- Регулярно обновляйте зависимости

Результат

Безопасное, надёжное и предсказуемое REST API

HTTP/1.1 200 OK

```
{
  "success": true,
  "data": { "id": 1, "name": "Order" }
}
```

Безопасные ошибки и logging

- Клиенту возвращайте понятную ошибку без stack trace и внутренних SQL/Prisma деталей.
- Validation failures полезно логировать как security signal.
- Не логируйте passwords, JWT, session cookies и другие credentials.
- Повторяющиеся validation failures или 429 могут указывать на abuse.
- В production подробности ошибки остаются в server logs.

Мини-практика: hardening Product API

Добавьте validation, body limit, Helmet и rate limiting в Internet Shop API.

Задание

- Установить express-validator, helmet и express-rate-limit
- Создать validateRequest middleware
- Создать validation rules для Product
- Проверить id в params
- Использовать matchedData
- Ограничить JSON body
- Подключить Helmet
- Добавить общий API limiter
- Добавить более строгий login limiter
- Проверить ответы 400, 413 и 429

Критерии проверки

- Некорректный Product не попадает в Service
- Неожиданные поля не передаются в Prisma
- Слишком большой request отклоняется
- Security headers присутствуют
- Частые login requests получают 429
- Нормальный CRUD продолжает работать

Контрольные вопросы

1. Почему client-side validation недостаточна для безопасности?
2. Чем syntactic validation отличается от semantic validation?
3. Почему allowlist предпочтительнее denylist?
4. Для чего используется validationResult()?
5. Что возвращает matchedData()?

6. Почему опасно передавать req.body целиком в Prisma?
7. Для чего нужен request body limit?
8. Что делает Helmet?
9. Когда сервер возвращает 429 Too Many Requests?
10. Почему rate limiting не заменяет Authentication и Authorization?

Итоги занятия

- 1 Все входные HTTP-данные считаются недоверенными.
- 2 Server-side validation обязательна независимо от frontend checks.
- 3 express-validator проверяет body, params и query.
- 4 validationResult() централизует validation errors.
- 5 matchedData() помогает использовать только проверенные поля.
- 6 Validation дополняется Service rules и database constraints.
- 7 Request body size limit уменьшает риск resource exhaustion.
- 8 Helmet добавляет browser security response headers.
- 9 Rate limiting ограничивает abuse и online brute-force attempts.
- 10 Безопасный REST API строится как несколько независимых защитных слоёв.